WIFI_RIG_CTRL / M5CoreHamCAT

Raspberry Pi セットアップガイド

Raspberry Pi Imager での SD カード作成から、サーバー構築、アプリ接続、外出先からの接続（WireGuard）までの手順です。

1. システム構成と準備するもの
2. SD カードのイメージ作成
3. 初回起動と SSH 接続
4. 無線機の接続
5. サーバーのセットアップ
6. 動作確認
7. アプリから接続する
8. 任意の設定（API Key・IP 固定・VPN・VNC）
9. 更新方法
10. トラブルシューティング

対象バージョン：Pi API v3.17 以降（GitHub 公開版 v3.17）
動作確認リグ：Yaesu FT-991A　／　推奨 Pi：Raspberry Pi 4 / 5（メモリ 1GB 以上）／ FT8 を使わない場合は Zero 2 W も可
GitHub：https://github.com/ji1ore/M5CoreHamCAT　／　JI1ORE

SAFETY NOTICE / v3.19 追補

! 安全上の注意 — 誤送信（意図しない送信）を防ぐために

Wifi_Rig_CTRL は、離れた場所から無線機を送信させる仕組みです。Raspberry Pi・ネットワーク・電源に異常が起きると、 **無線機が送信し続ける**おそれがあります。実際に、Pi の不調で FT-991A が送信を続けた事例がありました。 ソフトウェアの安全装置だけに頼らず、次の対策を必ず行ってください。

必ず行ってください

1. **無線機の TOT（送信タイムアウトタイマー）を設定する。** FT-991A ではメニューの「**TX TOT**」を **3〜5 分**にします（初期値は OFF）。 Pi がどんな状態になっても、無線機が自分で送信を止める**最後の砦**です。ほかの機種でも、送信タイマーの機能を設定してください。
2. **PTT 方式は CAT（RIG）を使う。** アプリまたは M5Core の PTT 設定で「RIG」を選びます。RTS/DTR 方式では、Pi がフリーズすると USB シリアルの信号が ON のまま残り、 ソフトウェアでは止められません。無線機側の RTS/DTR による送信設定（PC KEYING など）も OFF にしておくと安心です。
3. **異常に気づいたら、次の順で止める。** ① 無線機の電源を切る → ② Pi と無線機をつなぐ USB ケーブルを抜く → ③ Pi の電源を抜く。 原因の調査は、安全を確保してから行います。

誤送信の主な原因と対策

原因	対策
Pi のフリーズ・再起動の繰り返し（電源不足、SD カードの劣化）	公式電源を使い、`vcgencmd get_throttled` が `throttled=0x0` になることを確認します。信頼性の高い microSD カードを使います。
送信中の回り込み（電波が USB ケーブルに乗り、CAT や USB オーディオが切断される）	USB ケーブルにフェライトコアを付け、同軸ケーブルのコモンモード対策をします。最初は送信出力を下げて動作を確認します。
Wi-Fi・VPN の切断	アプリからの通信が 5 秒途絶えると、Pi が自動で送信を OFF にします。ただし、rigctld が応答しない状態では止められない場合があるため、TOT を併用します。
無人での自動送信（FT8 の CQ AUTO など）	送信状態を見張れるときだけ使います。CQ AUTO は、止めるまで周期ごとに送信を続けます。

ログを残す設定（推奨）

初期設定では、Pi を再起動するとログが消えます。誤送信の原因を後から調べられるよう、ログを保存する設定にしておきます。

```
sudo mkdir -p /var/log/journal
sudo systemctl restart systemd-journald
# 前回起動時のログを見る
sudo journalctl -u fastapi -b -1 --no-pager | tail -100
```

※ FT-991A の TOT のメニュー番号は、取扱説明書のメニュー一覧で「TX TOT」を確認してください。

1 システム構成と準備するもの

Raspberry Pi で Hamlib（rigctld）を FastAPI でラップし、Android／iOS アプリや M5Core から Wi-Fi 経由で無線機を操作します。



必要なもの	備考
Raspberry Pi 4 / 5 （メモリ 1GB 以上）	**推奨機種**です。FT8 のデコードは処理が重いため、Pi 4 / 5 を使ってください。USB-A ポートが 4 つあるので、無線機や CW サーバー（ATOM Lite）をそのままつなげて使いやすいです。 **FT8 を使わない場合**（CAT 操作、音声、APRS、CW、RTTY だけを使う場合）は、**Raspberry Pi Zero 2 W** でも動作します。
microSD カード	16GB 以上で、信頼性の高いもの。Wifi_Rig_CTRL 専用に新しく書き込みます。
USB ケーブル類	Pi の電源用と、無線機との接続用です。電源は、Pi 4 には 5V 3A（USB-C）、Pi 5 には 5V 5A（USB-C・27W）の公式電源を使ってください。Zero 2 W はデータ用ポートが micro-USB（OTG）なので、変換アダプタが必要です。
Wi-Fi ルーター	Pi とアプリの端末を同じネットワークにつなぎます。
PC（Mac / Windows）	SD カードの作成と SSH 接続に使います。
操作端末	スマートフォン（Android / iPhone）に「Wifi Rig CTRL」アプリを入れて使います。M5CoreS3SE / Core2 / Core2 Tough でも操作できます。

⚠ **Wifi_Rig_CTRL 専用の Raspberry Pi で使ってください（ほかのアプリとの共存は非推奨）**
同じ Raspberry Pi でほかのアプリやサービスを動かすことは推奨しません。新しく書き込んだ SD カードで、Wifi_Rig_CTRL だけを動かしてください。理由は次のとおりです。
・USB オーディオと CAT 用のシリアルポートを Wifi_Rig_CTRL が使い続けるため、ほかのアプリ（WSJT-X、fldigi、別の rigctld など）と取り合いになります。
・ポート 8000 / 50000 / 8888 / 8889 を使うため、同じポートを使うアプリとは同時に動きません。
・セットアップスクリプトが、Hamlib の入れ替え（/usr/local）、systemd サービスの登録、sudoers の設定、IPv6 の無効化（setup_netwk.sh）など、システム全体の設定を変更します。
・FT8 のデコードは CPU を多く使うため、ほかの処理があると取りこぼしや遅れの原因になります。

Raspberry Pi Zero 2 W を使う場合：USB ポートが足りないので工夫が必要です
Zero 2 W でデータ通信に使える USB ポートは、micro-USB（OTG）の **1 つだけ**です（もう 1 つは電源専用）。無線機と CW サーバー（ATOM Lite、Wifi_Rig_CW を使う場合）の両方をつなぐには、次のような工夫が必要です。
・micro-USB（OTG）対応の **USB ハブ**を使い、無線機と ATOM Lite をハブにつなぐ（電源付きのハブだと、動作が安定します）。
・ハブとケーブルは、データ通信に対応したものを使う（充電専用のケーブルでは認識されません）。
この点からも、USB-A ポートが 4 つある Raspberry Pi 4 以上のほうが使いやすく、おすすめです。

2 SD カードのイメージ作成

PC に **Raspberry Pi Imager** をインストールします（https://www.raspberrypi.com/software/）。

1. microSD カードを PC に挿して、Raspberry Pi Imager を起動します。
2. **デバイス**で使用する機種（「Raspberry Pi 4」「Raspberry Pi 5」「Raspberry Pi Zero 2 W」など）を選びます。
3. **OS**で「Raspberry Pi OS (other)」→「**Raspberry Pi OS Lite (64-bit)**」を選びます。
   標準は Lite を選び、セットアップで VNC「なし」を選ぶ構成です。Raspberry Pi Zero 2 W で Lite を使う場合は、VNC は必ず「なし」を選んでください（5 章・8-5 を参照）。
4. **ストレージ**で書き込み先の microSD カードを選びます。
5. 「次へ」→「**設定を編集する**」（OS のカスタマイズ）で、次の表のとおりに設定します。**特に Wi-Fi の設定を忘れないでください。**
6. 保存して「はい」を押すと書き込みが始まります。完了したらカードを取り出します。

項目	設定値	説明
ホスト名	任意（例：`raspizero`）	ここで決めた名前に `.local` を付けた名前（例：`raspizero.local`）で、SSH やアプリから接続できるようになります。**このあとの手順で使うので、控えておいてください。**本書では例として `raspizero` を使います。
ユーザー名 / パスワード	任意	スクリプトはユーザー名に依存しません（`pi` でなくても動作します）。SSH で使うので控えておいてください。
★ Wi-Fi（SSID / パスワード） 必ず設定する	**自宅ルーターの値**	**「無線 LAN を設定する」**にチェックを入れて入力します。Wi-Fi の国は **JP** にします。SSID は大文字・小文字を区別します。
ロケール	Asia/Tokyo ／ jp	タイムゾーンとキーボード配列。FT8 は時刻が正確である必要があります。
SSH	**有効にする**	「パスワード認証を使う」を選びます。

> **⚠ Wi-Fi の設定を忘れずに**
> Wi-Fi を設定しないで書き込むと、Pi がネットワークにつながらず、SSH でもアプリからも接続できません（Lite 版には画面がないので、Pi 側で設定し直すのも大変です）。書き込む前に、次の 3 点を確認してください。
> ・「無線 LAN を設定する」にチェックが入っている
> ・SSID とパスワードが正しい（大文字・小文字の違いに注意）
> ・Wi-Fi の国が **JP** になっている
> 設定を忘れた場合は、Imager で SD カードを作り直すのが一番かんたんです（Pi 4 / 5 なら、有線 LAN でつないで `sudo nmtui` で設定することもできます）。

> **注意**
> Raspberry Pi Zero 2 W の Wi-Fi は **2.4GHz のみ**対応です。5GHz 専用の SSID には接続できません（Pi 4 / 5 は 5GHz にも対応しています）。Pi 4 / 5 は、有線 LAN でつなぐとさらに安定します。

3 初回起動と SSH 接続

1. microSD カードを Pi に挿して、電源をつなぎます。初回の起動には時間がかかります（**目安 3 分**）。
2. PC のターミナル（Mac は「ターミナル」、Windows は PowerShell）から SSH で接続します。

> ⚠ **Raspberry Pi Zero 2 W は、初回の起動にとても時間がかかります**
> Raspberry Pi Imager で書き込んだ SD カードで初めて起動すると、Imager で設定したホスト名・ユーザー・Wi-Fi などの初期設定が自動で行われ、途中で何度か再起動します。Zero 2 W は処理が遅いため、Pi 4 / 5 よりも時間がかかります。**目安は 3 分**です。
> ・3 分たつまでは、SSH で接続できなくても**電源を抜かずに待ってください**。途中で電源を切ると、初期設定が終わらないことがあります。
> ・**3 分以上たっても接続できない場合は、Raspberry Pi Imager での SD カードの書き込みからやり直してください**（2 章）。Wi-Fi の設定（SSID・パスワード・国）も、もう一度確認してください。
> ・本体の緑の LED の点滅が落ち着いてから、SSH で接続してみてください。

```
ssh <ユーザー名>@<ホスト名>.local      # 例：ssh pi@raspizero.local
```

`<ユーザー名>` と `<ホスト名>` は、手順 2 で Imager に設定したものです。

初回は `Are you sure you want to continue connecting (yes/no)?` と聞かれるので、`yes` と入力してから、パスワードを入力します。

> **.local で接続できないとき**
> ルーターの管理画面で Pi の IP アドレスを調べて、`ssh <ユーザー名>@192.168.x.x` で接続します。

> **「REMOTE HOST IDENTIFICATION HAS CHANGED!」と表示されたとき**
> SD カードを作り直すと SSH の鍵が変わるため、PC に残っている古い鍵とぶつかってこのエラーが出ます。自分で SD カードを作り直した場合は、古い鍵を削除してから接続し直してください。
>
> ```
> ssh-keygen -R <ホスト名>.local   # 例：raspizero.local
> ssh-keygen -R 192.168.x.x   # IP アドレスで接続していた場合
> ```

4 無線機の接続

セットアップを実行する**前に**、Pi と無線機を USB ケーブルでつなぎ、無線機の電源を入れておいてください。

- セットアップスクリプトは、つながっている USB オーディオデバイスを**自動で検出**し、音声と Direwolf（APRS）の設定に書き込みます。
- USB オーディオが複数見つかったときは、番号を選ぶよう聞かれます。
- FTDI と CH340 の USB シリアルドライバーは、スクリプトが自動で読み込みます。

接続できているかは、次のコマンドで確認できます。

```
arecord -l            # 「USB Audio」が表示されれば OK
ls /dev/ttyUSB* /dev/ttyACM*   # CAT 用のシリアルポート
```

5 サーバーのセットアップ

方法は 2 つあります。**方法 A（アプリから実行）**がかんたんで、おすすめです。M5Core だけを使う場合や、コマンドで入れたい場合は、方法 B を使います。

方法 A：アプリから実行する（「SETUP Pi」／「Pi Setup (SSH)」）

対応バージョン：Android / iOS アプリ v3.16 以降。スマートフォンを、Pi と同じ Wi-Fi につないでおいてください。

アプリが SSH で Pi にログインし、アプリに入っているセットアップ用ファイル一式を転送して、`setup_fastapi_radio.sh` を実行します。**Pi に SSH で接続したり、GitHub からダウンロードしたりする必要はありません。**

1. **アプリをインストールします。**QR コードを読み取るか、ストアで「**Wifi Rig CTRL**」を検索してインストールします。

Android：Google Play
「Wifi Rig CTRL」で検索
play.google.com/store/apps/details?id=com.ji1ore.wifi_rig_ctrl

iPhone / iPad：App Store
「Wifi Rig CTRL」で検索
apps.apple.com/jp/app/wifi-rig-ctrl/id6788414941

2. **セットアップ画面を開きます。**

	Android	iOS
画面の開き方	接続設定画面の右上にある **「⋮」** →**「Setup Pi (SSH)」**	接続設定画面の右上にある **「⋯」**（ellipsis）→**「Pi Setup (SSH)」**
画面のタイトル	SETUP (SSH)	

3. **接続先を入力します。**

入力欄	入力する値
IP / Host （Android：Pi IP Address）	`<ホスト名>.local`（Imager で設定したホスト名。例：`raspizero.local`）、または Pi の IP アドレス（例：`192.168.1.17`）。 .local で接続できないときは IP アドレスを入力してください。
SSH User	Imager で設定したユーザー名。空欄にすると `pi` になります。
SSH Password	Imager で設定したパスワード。sudo の実行にも使われます。

4. **「VNC デスクトップ」を選びます。**「XFCE」「PIXEL」**「なし」**から選びます。**標準は「なし」**です。
Raspberry Pi Zero 2 W で OS が Lite の場合は、必ず「なし」を選んでください。デスクトップを入れると CPU とメモリに余裕がなくなり、CW のタイミングがずれる原因になります（8-5）。

5. **「Run Setup」をタップします。**画面の下にログが表示され、次の順に処理が進みます。
   - SSH で接続し、ファイルを転送します（api.py、create_api.sh、cw_bridge.py、setup_fastapi_radio.sh、set_api_key.sh）。
   - セットアップスクリプトを実行し、Hamlib、FastAPI、Direwolf をインストールします（**Zero 2 W では 1 時間以上**かかります。Pi 4 / 5 ではもっと早く終わります）。

6. **完了の表示を確認します。**「✓ **SETUP 完了 — 次に UPDATE 画面で「Update Pi」を実行してください**」と表示されたら完了です。ログの最後に `fastapi / fastapi-audio / direwolf : OK` と表示されているかも確認してください。

7. **「Update Pi」を実行します。**Pi に接続してから、次の画面で実行します。FT8 用の **mfsk-decode のビルドが自動で始まります（Zero 2 W では 30〜60 分）**。FT8 を使わない場合でも、Pi 側の API を最新にするために実行してください。

- Android：リグ選択画面の「**UPDATE**」→「**Update Pi**」
- iOS：「…」→「**Admin**」→「**Update Pi API**」

ビルドの進み具合は Admin 画面のセットアップログで確認できます。ビルドが終わるまで、FT8 はデコードされません。

実行中の注意

・**所要時間（Raspberry Pi Zero 2 W）：SETUP は 1 時間以上、続けて行う Update Pi まで含めると約 1 時間 30 分**かかります。時間に余裕をもって始めてください。
・処理が終わるまで、**画面を消したり、ほかの画面に移動したりしないでください。**途中で接続が切れると、セットアップが中断します。スリープしないように、充電しながら実行するのがおすすめです。
・Pi と無線機を USB でつなぎ、無線機の電源を入れた状態で実行してください（手順 4）。
・Pi がインターネットに接続できる必要があります。

うまくいかないとき

・「接続中...」のまま進まない、またはエラーになる：IP アドレスで入力し直してください。スマートフォンと Pi が同じ Wi-Fi につながっているかも確認してください。
・「⚠ セットアップ失敗 (exit …)」と表示される：もう一度「Run Setup」を実行してください（何度実行しても問題ありません）。それでも失敗する場合は、方法 B で SSH から実行すると、エラーの内容を確認しやすくなります。
・アプリは Pi の SSH の鍵を確認しないため、SD カードを作り直したあとでも、PC のような鍵の警告は出ません。

方法 B：SSH でコマンドを実行する

手順 3 のとおり SSH でログインしてから、次のコマンドを実行します。完了したら `sudo reboot` で再起動します。インターネット接続が必要です。

B-1. ネットワーク設定（任意・Zero 2 W は実行を推奨）

IPv6 を無効にして、接続を安定させます。

```
wget https://raw.githubusercontent.com/ji1ore/M5CoreHamCAT/main/v3.17/RaspberryPiSetup/setup_netwk.sh
chmod +x setup_netwk.sh
bash setup_netwk.sh
```

B-2. 環境構築（FT8 を含むすべての機能）

```
BASE=https://raw.githubusercontent.com/ji1ore/M5CoreHamCAT/main/v3.17/RaspberryPiSetup
wget $BASE/setup_fastapi_radio.sh
wget $BASE/create_api.sh
wget $BASE/set_api_key.sh
wget $BASE/api.py
wget $BASE/cw_bridge.py
chmod +x setup_fastapi_radio.sh create_api.sh set_api_key.sh
sudo bash setup_fastapi_radio.sh none     # none = VNC なし（標準）
```

VNC の選択（最後の引数）

`none`（VNC なし・標準）／ `xfce` ／ `pixel` から選びます。**引数を付けないと XFCE が入ります。**標準の「VNC なし」にするには、必ず `none` を付けてください。Raspberry Pi Zero 2 W で Lite の場合も `none` にします。

補足

・必ず `sudo` を付けて実行してください（systemd サービスの登録に必要です）。
・Hamlib のビルドには時間がかかります（Zero 2 W では特に長くかかります）。途中で止まったように見えても、そのまま待ってください。
・`api.py` を同じフォルダに置いておくと、最初から最新版が適用されます。
・同じスクリプトを何度実行しても問題ありません。途中で失敗したときは、もう一度実行してください。
・最後に `fastapi / fastapi-audio / direwolf : OK` と表示されたら、`sudo reboot` で再起動します。

スクリプトの処理内容：パッケージのインストール → USB シリアルドライバー → Hamlib 4.7.2 のビルド → Python 仮想環境と FastAPI → Direwolf → オーディオの検出 → systemd サービスの登録 → sudoers の設定 → api.py の適用とサービスの起動

6 動作確認

再起動したら、もう一度 SSH でログインして確認します。

```
# サービスの状態（すべて active（running）なら OK）
sudo systemctl status fastapi fastapi-audio direwolf

# API のバージョン
curl http://localhost:8000/version

# ログをリアルタイムで表示（Ctrl+C で終了）
sudo journalctl -u fastapi -f
sudo journalctl -u fastapi-audio -f
```

ポート	用途
8000	メイン API（CAT 制御・APRS・FT8 デコード結果）
50000	音声ストリーミング（fastapi-audio）
8888 / 8889	CW（M5 Server UDP / cw_bridge）

7 アプリから接続する

アプリ（Google Play / App Store の「Wifi Rig CTRL」）または M5Core の **RIG CONNECT** 画面で、次のように設定します。

項目	設定値
ホスト名	`<ホスト名>.local`（Imager で設定したホスト名。例：`raspizero.local`）、または IP アドレス
API Port	`8000`
Audio Port	`50000`
CAT デバイス	無線機がつながっているシリアルポート（例：`/dev/ttyUSB0`）。違うものを選ぶと接続できません。

M5Core のファームウェアは、M5Stack の **WebBurner**（https://burner.m5stack.com）で書き込みます。Chrome または Edge で開き、「探す」で「**Wifi_Rig_CTRL**」を検索して、機種に合ったもの（例：Wifi_Rig_CTRL For M5Core2）を選び、「デバイスに書き込む」を押します。M5Burner（デスクトップ版）は 2027 年 1 月 1 日で提供が終わります。

8 任意の設定

8-1. API Key による認証

> **WireGuard の設定をアプリから行う場合は、ここでの作業は不要です**
> アプリの「WireGuard Setup」画面（8-3 の方法 A）で、API Key も一緒に設定されます。キーは自動で作られ、Pi とアプリの両方に保存されます。

LAN 内だけで使う場合や、WireGuard をスクリプトで設定する場合は、Pi に SSH でログインして、次のコマンドで設定します。

```
bash ~/set_api_key.sh あなたのシークレットキー    # 設定する
bash ~/set_api_key.sh ""                          # 無効にする
```

設定したら、アプリの接続設定画面の「API Key」にも同じキーを入力します。外出先から接続する場合は、API Key の設定が**必須**です。

8-2. IP アドレスの固定

IP アドレスが変わると、アプリの接続先を変える必要があり、SSH の鍵の警告も出やすくなります。ルーターの DHCP 予約で固定するのがかんたんです。Pi 側で固定する場合は、次のコマンドを使います。

```
nmcli connection show                      # 接続名を確認
sudo nmcli connection modify "接続名" ipv4.method manual \
  ipv4.addresses 192.168.1.50/24 ipv4.gateway 192.168.1.1 ipv4.dns "192.168.1.1 8.8.8.8"
sudo nmcli connection up "接続名"
```

8-3. 外出先から接続する（WireGuard VPN）

自宅の LAN 内だけで使う場合は不要です。どちらの方法でも、先に次の 3 つを準備しておきます。

- **スマートフォンに WireGuard アプリをインストール**しておきます。Google Play / App Store で「WireGuard」を検索します。**入れていないと、設定をインポートできません。**
- DuckDNS（https://www.duckdns.org/）でサブドメインを作り、**トークン**を控えておきます。
- ルーターで **UDP 51820** を Pi の IP アドレスに転送します（Pi の IP アドレスは固定しておきます）。

方法 A：アプリから設定する（おすすめ）

自宅の Wi-Fi で、アプリが Pi に接続できる状態にしてから行います。

1. 接続設定画面の右上のメニュー（Android「⋮」／ iOS「…」）→「**WireGuard Setup**」を開きます。
2. 次の項目を入力します。

項目	入力する値
Pi の IP アドレス	Pi の LAN 内の IP アドレス
固定 IP（任意）	「Pi の IP を固定する」をオンにすると、指定した IP アドレスに固定します。
DNS ドメイン	DuckDNS のサブドメイン名（例：`myhome` → myhome.duckdns.org）
DuckDNS トークン	DuckDNS のトークン。入力欄の横にあるカメラのボタン（Android「📷」／ iOS のカメラアイコン）を押して、PC などに表示した **DuckDNS のアカウント画面（トークンが表示されている画面）をカメラで写す**と、トークンを読み取って入力できます。
API キー（必須）	**自動で作られて、最初から入力されています。**そのままで構いません。好きなキーにしたい場合は上書きします。
Pi パスワード（sudo 用）	Pi のログインパスワード

3. 「**WireGuard セットアップ開始**」をタップします。WireGuard のインストールと設定のあと、**API キーが Pi に設定され、アプリにも自動で保存されます**（接続設定画面の「API Key」と同じ値になります）。
4. 完了したら「**WireGuard アプリにインポート**」をタップして、WireGuard アプリにトンネルを追加します。
5. 外出先では、WireGuard のトンネルをオンにしてから、接続先ホストを `10.0.0.1` にして接続します（VPN 経由では `.local` の名前が使えません）。

方法 B：スクリプトで設定する

Pi で次のスクリプトを取得し、冒頭の `STATIC_IP`、`GATEWAY`、`DUCKDNS_DOMAIN` を書き換えてから実行します。画面に表示される QR コードを WireGuard アプリで読み取ります。この方法では API Key は設定されないので、8-1 のコマンドで別に設定してください。

```
wget https://raw.githubusercontent.com/ji1ore/M5CoreHamCAT/main/v3.10/WireGuard/setup_wireguard.sh
chmod +x setup_wireguard.sh
nano setup_wireguard.sh      # 変数を編集して保存
bash setup_wireguard.sh
```

8-4. FT8 送信用バイナリ（自動ビルドに失敗したときだけ）

```
wget https://raw.githubusercontent.com/ji1ore/M5CoreHamCAT/main/v3.17/RaspberryPiSetup/setup_ft8_encode.sh
bash setup_ft8_encode.sh
```

このバイナリがなくても、FT8 の受信とデコードはできます。

8-5. VNC（GUI デスクトップ）を使う場合

セットアップのときに、VNC のデスクトップを「なし／XFCE／PIXEL」から選びます（5 章）。

選択	内容
なし（標準）	VNC もデスクトップも入れません。操作は SSH とアプリから行います。CPU に余裕があり、CW などのタイミングが安定するので、**通常はこちらを選んでください。** **Raspberry Pi Zero 2 W で OS が Lite の場合は、必ず「なし」を選んでください。**
XFCE	VNC と XFCE デスクトップを入れます。
PIXEL	VNC と Raspberry Pi 標準のデスクトップ（PIXEL / LXDE）を入れます。

以下は XFCE / PIXEL を選んだ場合の説明です。「なし」の場合は、この節の作業は不要です。

> ⚠ **X（デスクトップ）を常に動かしたままにしないでください**
> X（GUI のデスクトップ）が動いていると CPU が占有され、**CW の信号のタイミングがずれます**（符号の長さや間隔が崩れます）。Wifi_Rig_CTRL を使っている間は、X と併用しないでください。
> そのため、セットアップでは Pi が常にコンソール（GUI なし）で起動するように設定し、**VNC を使うときだけデスクトップを起動**します。作業が終わったら、必ず止めてください。

1. デスクトップと VNC は、セットアップのあとにバックグラウンドで入ります（Zero 2 W では時間がかかります。進み具合は `~/ondemand_gui_setup.log` で確認できます）。
2. GUI を使いたいときに、SSH で次のコマンドを実行して VNC を起動します。

   ```
   sudo systemctl start vncserver@1
   ```

3. PC やタブレットの **VNC ビューア**（RealVNC Viewer、TigerVNC Viewer など）で接続します。

項目	設定値
接続先	`<Pi の IP アドレス>:5901`（例：`192.168.1.17:5901`）
パスワード	`raspberry`

   選んだデスクトップ（PIXEL を選んだ場合は LXDE のデスクトップ）が表示されます。
4. 使い終わったら、VNC を止めます（止めるまでデスクトップが動き続けます）。

   ```
   sudo systemctl stop vncserver@1
   ```

外出先から WireGuard 経由で接続する場合は、`10.0.0.1:5901` を指定します。VNC のパスワードは、必要に応じて変更してください（初期値のままだと、同じネットワークの人なら誰でも接続できます）。

9 更新方法

方法	手順
アプリ（推奨）	UPDATE 画面の「**Update Pi**」ボタンを押すと、api.py の転送、create_api.sh の実行、再起動、mfsk-decode のビルドが自動で行われます。Zero 2 W では 30〜60 分かかります。完了まで電源を切らないでください。
GitHub から	`wget -O ~/create_api.sh https://raw.githubusercontent.com/ji1ore/M5CoreHamCAT/main/v3.17/RaspberryPiSetup/create_api.sh` `sudo bash ~/create_api.sh`
アプリ本体	Google Play / App Store から更新します。アプリを更新したら、続けて「Update Pi」も実行して、Pi 側を同じバージョンに揃えてください。
Hamlib	アプリの「Update Hamlib」ボタン、または `install_hamlib.sh` を実行します。

10 トラブルシューティング

症状	対処
SSH で「REMOTE HOST IDENTIFICATION HAS CHANGED!」と表示される	`ssh-keygen -R <ホスト名または IP>` で古い鍵を削除してから、接続し直します（手順 3 を参照）。
`<ホスト名>.local` が見つからない	IP アドレスで接続します。Wi-Fi が 2.4GHz か、SSID とパスワードが正しいかを確認してください。
サービスが NG / failed になる	`sudo journalctl -u fastapi -n 100` でエラーを確認し、`sudo bash setup_fastapi_radio.sh` をもう一度実行します。
無線機の情報が取れない	CAT デバイスの選択と、無線機の CAT ボーレートの設定を確認します。直らない場合は `sudo reboot` で再起動します。
音声が聞こえない	`arecord -l` で USB Audio が見えるか確認します。無線機をつながずにセットアップした場合は、無線機をつないだ状態でセットアップをもう一度実行します。
FT8 がデコードされない	mfsk-decode をビルド中の可能性があります（Zero 2 W では 30〜60 分）。アプリの Admin 画面で進捗を確認するか、「Update Pi」を実行します。
音声が数秒途切れる	遅延を防ぐために、10 分ごとに再接続しているためです。異常ではありません。

本書は Wifi_Rig_CTRL リポジトリの readme.txt、RaspberryPiSetup/readme.txt、WireGuard/readme.txt をもとに作成しました。最新情報は GitHub を参照してください。